

Trabalho Criptografia - Chat P2P

Mateus Lima Rodrigues, 536334

Introdução:

O projeto consiste em uma aplicação chat seguro cliente-servidor desenvolvida em python. Dividi em dois arquivos para melhorar na organização:

- ***modos_operacao.py***: Aqui é o arquivo que contém toda a parte da lógica criptográfica, nesse arquivo estão as funções de geração de chave RSA, serialização, padding e a parte principal do trabalho, a implementação manual dos modos de operação do AES, CBC e o CTR
- ***chat.py***: Agora este é o arquivo que gerencia toda a camada de redes, a interface do usuário com menu e o chat em si, e o fluxo do handshake entre os dois usuários

Para executar, é necessário ter o python instalado e também a biblioteca cryptography que vai ajudar com os métodos primitivos, para instalar, execute:

pip install cryptography

Para rodar corretamente, abra duas instâncias do terminal, no primeiro escolha a opção 1(servidor) e no segundo escolha a opção 2(cliente), e escolha o modo de operação da sua escolha

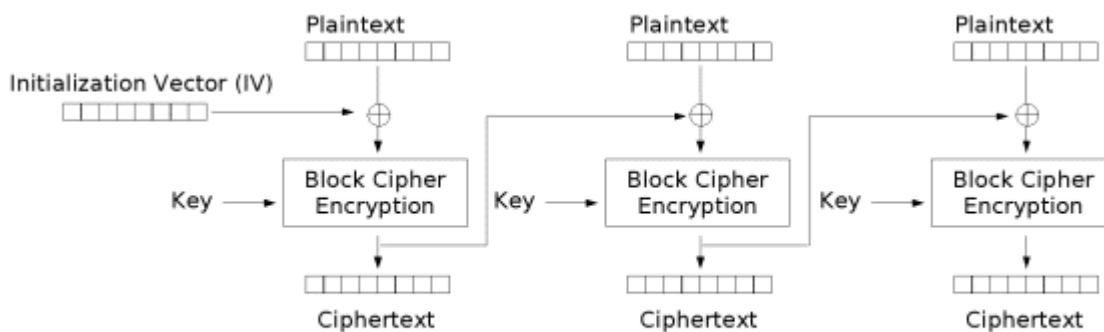
Decisões de Projeto:

Como algoritmo assimétrico, foi escolhido o RSA, escolhi um tamanho de chave de 2048 bits que é o padrão atual para garantir segurança, cada execução no servidor gera um par de chaves RSA, pública e privada, a pública é enviada ao cliente, o cliente gera uma chave de sessão simétrica aleatória de 32 bytes, então a chave é cifrada usando o OAEP no SHA 256, por fim o servidor descriptografar a chave de sessão com a chave privada

Após esse handshake, o resto é cifrado usando o AES, usamos ele como cifrador de blocos base, usando o ECB internamente para gerar a base perfeita para o projeto, para assim gerar a implementação completa dos modos CBC e CTR

Modos de Operação:

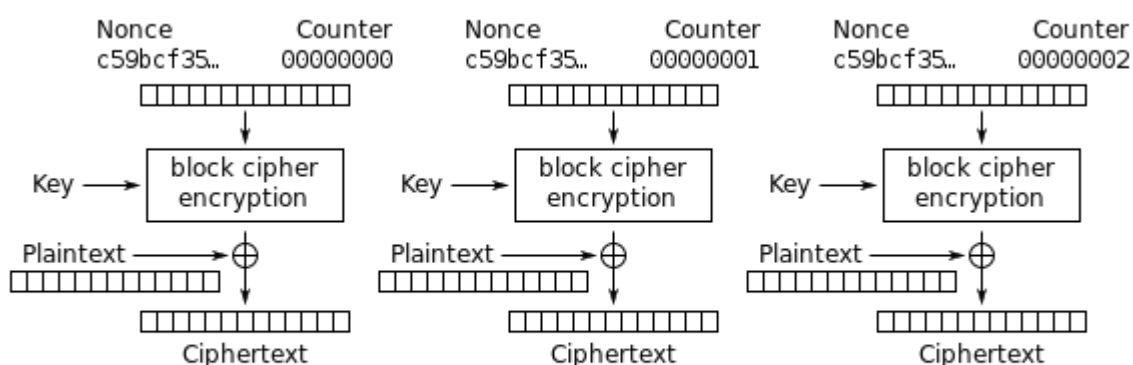
Para as imagens dos modos, peguei do site embarcados.com



Cipher Block Chaining (CBC) mode encryption

Aqui temos o esquema geral do modo CBC, nesse modo, cada bloco de texto plano é combinado com o bloco cifrado anterior antes da cifragem, para implementar, precisamos:

- Um IV, que é um vetor de inicialização de 16 bytes, usado para adicionar uma camada de aleatoriedade no primeiro bloco de cifra, cada nova mensagem gera um novo IV, garantindo a segurança
- A camada de padding foi implementada pensando no modelo PKCS#7, uma vez que o AES opera em blocos de 16 bytes e sendo mensagens de texto, dificilmente teríamos múltiplos exatos do bloco
- Como mostrado na imagem, cada bloco de texto plano sofre um XOR com um bloco cifrado anteriormente, antes de passar pela encriptação



Counter (CTR) mode encryption

Agora para o CTR, não precisamos de padding como o anterior, pois tudo vai virar uma cifra de fluxo no final

- Aqui teremos um nonce aleatório de 8 bytes para cada mensagem, depois desses 8 bytes de nonce temos mais 8 representando o contador, cada nonce é usado apenas uma vez, depois disso é gerado outro aleatório para a próxima mensagem

- Por fim, nonce é enviado junto da mensagem cifrada para que assim, o receptor possa reconstruir o contador original

Conclusão:

Para finalizar, podemos comparar os dois modos:

Característica	Modo CBC	Modo CTR
Dificuldade	Média devido aos problemas que podem acontecer relacionados ao padding	Baixa, há facilidade de encriptação e decriptação, uma vez que a lógica é a mesma
Padding	Obrigatório, a mensagem deve ser obrigatoriamente múltipla de 16 bytes	Não usa, uma vez que ao final de tudo ele vai funcionar como uma cifra de fluxo
Inicialização	Usa o IV no XOR do bloco 1	Usa o nonce em todo o processo
Performance	Serial, uma vez que precisamos obrigatoriamente fazer bloco a bloco, não podemos partir para o bloco 3 sem antes fazer o 2 e o 1	Altamente paralelo, uma vez que assim como o modo ECB, podemos escolher quais blocos precisamos
Integridade	Um erro afeta todos os blocos a seguir(pesadelo)	Erro em um bit vai afetar apenas aquele bit no texto final

Basicamente, o trabalho foi ótimo para ver como funcionam os modos de operação de forma mais aprofundada, inclusive motivando mais a estudar, procurar outras alternativas fora o livro, durante o estudo e implementação, essas foram minhas referências principais:

- [Canal Computerphile](#)
- [Site Embarcados](#)
- [Documentação Cryptography.io](#)